

PROJETO INTEGRADOR

RELATÓRIO DE REFATORAÇÃO E APLICAÇÃO DOS PRINCÍPIOS SOLID

Sistema Soluções Engenharia C&F

Christian Eric Barrantes Briceño

1. Introdução

Este relatório apresenta a revisão e a refatoração do sistema desktop Soluções Engenharia C&F, desenvolvido originalmente com Java Swing, JDBC e MySQL. O trabalho teve como finalidade preparar as classes para reaproveitamento em uma futura aplicação web, separando as regras de negócio da interface gráfica e do acesso ao banco de dados.

A atividade foi desenvolvida em um novo projeto Java no Apache NetBeans, preservando o projeto desktop anterior como evidência. A refatoração priorizou o princípio da responsabilidade única e adotou interfaces, injeção de dependências, tratamento específico de exceções e o padrão Repository.

2. Caracterização do sistema anterior

O sistema desktop oferece autenticação de usuários, cadastro e listagem de projetos, registro de auditorias e acompanhamento de não conformidades. A interface foi implementada em Java Swing e a persistência dos dados utiliza JDBC com banco MySQL.

Na versão anterior, parte das validações e das regras estava distribuída entre a classe Main, as telas Swing e as classes DAO. Essa organização criava dependência direta entre interface, persistência e regras de negócio, reduzindo a possibilidade de reutilização no futuro sistema web.

Elemento anterior	Problema identificado	Consequência
app.Main	Leitura de dados, conversões, validações, apresentação e chamadas ao DAO na mesma classe.	Método principal extenso e difícil de reaproveitar.
Telas e diálogos Swing	Instanciação direta de DAO e validações dentro dos eventos da interface.	Acoplamento entre a interface desktop e o banco de dados.
Classes DAO	Consultas SQL, mapeamento, tratamento de erro e apresentação com System.out.println.	Mistura entre persistência e apresentação.
ConnectionFactory	Usuário e senha do banco escritos diretamente no código.	Risco de segurança e baixa portabilidade.
Projeto.orcamento	Uso de double para valor monetário.	Possibilidade de imprecisão em cálculos decimais.
Métodos de busca/authenticação	Retorno null e ocultação de erros de SQL.	Maior risco de NullPointerException e falhas difíceis de diagnosticar.

3. Estrutura do projeto refatorado

O novo projeto foi dividido em pacotes conforme a responsabilidade de cada classe. A estrutura permite que as regras de negócio sejam chamadas por uma interface web sem depender das telas Swing.

Pacote	Responsabilidade	Principais classes
app	Inicialização e execução dos testes pelo método main().	Main
config	Criação e configuração das conexões com o banco.	ConnectionFactory
model	Representação das entidades do domínio.	Usuario, Projeto, Auditoria, NaoConformidade
repository	Contratos para as operações de persistência e exceção da camada.	UsuarioRepository, ProjetoRepository, AuditoriaRepository, NaoConformidadeRepository, RepositoryException
repository.jdbc	Implementações JDBC dos contratos de persistência.	UsuarioRepositoryJdbc, ProjetoRepositoryJdbc, AuditoriaRepositoryJdbc, NaoConformidadeRepositoryJdbc
service	Validações e regras de negócio independentes da interface.	UsuarioService, ProjetoService, AuditoriaService, NaoConformidadeService, ServiceException

4. Princípios SOLID aplicados

4.1 Princípio da Responsabilidade Única (SRP)

Cada classe passou a possuir um motivo principal para mudança. As entidades representam dados; os serviços concentram validações e regras; os repositórios definem operações de persistência; as implementações JDBC executam SQL; a ConnectionFactory cria conexões; e a classe Main apenas monta as dependências e executa testes.

Classes afetadas	Aplicação do SRP
ConnectionFactory	Responsável exclusivamente por obter configurações e abrir conexões.
ProjetoService, UsuarioService, AuditoriaService e NaoConformidadeService	Responsáveis pelas regras e validações do respectivo domínio.
Interfaces Repository	Responsáveis por definir os contratos de persistência.
Classes RepositoryJdbc	Responsáveis pelas instruções SQL e pelo mapeamento dos ResultSets.
Main	Responsável pela composição dos objetos e testes, sem regras internas de cadastro.

4.2 Princípio Aberto/Fechado (OCP)

Os serviços podem ser mantidos sem alteração quando for criada outra forma de persistência. Por exemplo, uma implementação em memória ou baseada em API poderá implementar ProjetoRepository, enquanto ProjetoService continuará utilizando o mesmo contrato. Assim, o comportamento pode ser estendido por novas implementações sem modificar as regras existentes.

4.3 Princípio da Segregação de Interfaces (ISP)

Foi criada uma interface específica para cada agregado do domínio. ProjetoService depende somente de ProjetoRepository; UsuarioService depende somente de UsuarioRepository; e o mesmo ocorre com auditorias e não conformidades. Dessa forma, nenhuma classe é obrigada a depender de métodos que não utiliza.

4.4 Princípio da Inversão de Dependência (DIP)

As classes de serviço dependem das abstrações do pacote repository, e não diretamente das classes JDBC. As implementações são recebidas pelo construtor, conforme exemplificado por ProjetoService(ProjetoRepository projetoRepository). Isso reduz o acoplamento e facilita o futuro uso dessas regras na camada web e em testes automatizados.

O princípio da substituição de Liskov não exigiu uma hierarquia específica nesta etapa. Contudo, as implementações JDBC respeitam os contratos das interfaces e podem ser substituídas por outras implementações compatíveis.

5. Refatorações e code smells tratados

Code smell/refatoração	Situação anterior	Solução aplicada	Classes/métodos afetados
Responsabilidades misturadas / Extract Class	Main e telas realizavam entrada, validação e persistência.	Criação das camadas service, repository e repository.jdbc.	Main, telas antigas, todos os Services e Repositories.
Acoplamento excessivo	Telas criavam ProjetoDAO, UsuarioDAO e outros DAOs diretamente.	Dependência de interfaces recebidas no construtor dos serviços.	ProjetoService, UsuarioService, AuditoriaService, NaoConformidadeService.
Long Method	Cadastro e conversões concentrados em Main e em métodos salvar() das telas.	Extração de métodos de validação e normalização em cada Service.	validarProjeto, validarUsuario, validarAuditoria, validarNaoConformidade.
Código duplicado	Conversões, validações e mensagens repetidas nas interfaces.	Centralização das regras nos serviços e de parâmetros/mapeamento nos repositórios.	preencherParametros(), mapearProjeto(), mapearUsuario(), mapearAuditoria(), mapearNaoConformidade().
Primitive Obsession	Orçamento representado por double.	Substituição por BigDecimal, compatível com DECIMAL(12,2) do banco.	model.Projeto e ProjetoRepositoryJdbc.
Retorno nulo	Autenticação e buscas podiam retornar null.	Uso de Optional nas interfaces e nos serviços.	Interfaces Repository, UsuarioRepositoryJdbc e métodos buscarPorId/buscarPorEmail/autenticar.
Tratamento inadequado de erro	DAO imprimia mensagem e continuava a execução.	Criação de RepositoryException e ServiceException.	Todos os repositórios e serviços.

Code smell/refatoração	Situação anterior	Solução aplicada	Classes/métodos afetados
Credenciais hard-coded	Senha do MySQL estava escrita em ConnectionFactory.	Leitura das variáveis DB_URL, DB_USUARIO e DB_SENHA.	config.ConnectionFactory.
Exposição de informação sensível	Objeto Usuario possuía senha e podia ser impresso.	toString sem senha e mapeamento de consultas sem carregar o hash.	model.Usuario e UsuarioRepositoryJdbc.
Recursos JDBC	Risco de conexões não fechadas em alterações futuras.	Uso sistemático de try-with-resources.	Todas as classes repository.jdbc.

6. Padrão de projeto adotado

6.1 Repository

Foi adotado o padrão Repository para criar uma fronteira entre as regras de negócio e a persistência. As interfaces do pacote repository descrevem operações como salvar, listarTodos, buscarPorId, atualizar e excluir. As classes do pacote repository.jdbc implementam essas operações com JDBC e MySQL.

A adoção do padrão foi justificada pela necessidade de preparar o sistema para a fase web. Um controlador web poderá chamar os serviços sem conhecer as consultas SQL, enquanto a persistência poderá evoluir sem alterar a regra de negócio.

6.2 Injeção de dependência por construtor

A implementação concreta é criada na composição da aplicação e fornecida ao serviço por meio do construtor. A abordagem torna as dependências explícitas, impede a criação de serviços sem repositório e melhora a testabilidade.

7. Testes realizados

Os testes foram implementados no método main(), conforme solicitado na atividade. O programa cria as implementações JDBC, injeta os repositórios nos serviços e executa consultas reais no banco importado.

Teste	Resultado observado
Conexão com o MySQL	Conexão realizada com sucesso utilizando o connector JDBC.
Listagem de usuários	Cinco usuários foram retornados, incluindo os tipos Administrador, Gerente, Consultor, Engenheiro e Cliente.
Autenticação	O usuário Christian Barrantes foi autenticado com credenciais válidas.
Listagem de projetos	Os projetos existentes no banco foram mapeados e exibidos, incluindo datas, orçamento, status e responsáveis.
Listagem de auditorias	Auditorias internas e externas foram retornadas, inclusive registros ainda não realizados.
Listagem de não conformidades	Os registros foram retornados com gravidade, prazo, responsável e status.
Validação negativa	Um Projeto vazio foi rejeitado pelo ProjetoService com a mensagem “O nome do projeto é obrigatório”.

O resultado apresentado pelo programa foi “TODOS OS TESTES FORAM CONCLUÍDOS!”, seguido de BUILD SUCCESSFUL. Os testes de consulta não modificaram o banco de dados.

8. Evidências do GitHub

Foi criado um repositório público para o novo projeto, disponível em: <https://github.com/ChrisEricB/SolucoesEngenhariaCFRefatorado>



Figura 1 – Página principal do repositório, com os diretórios e o README do projeto.

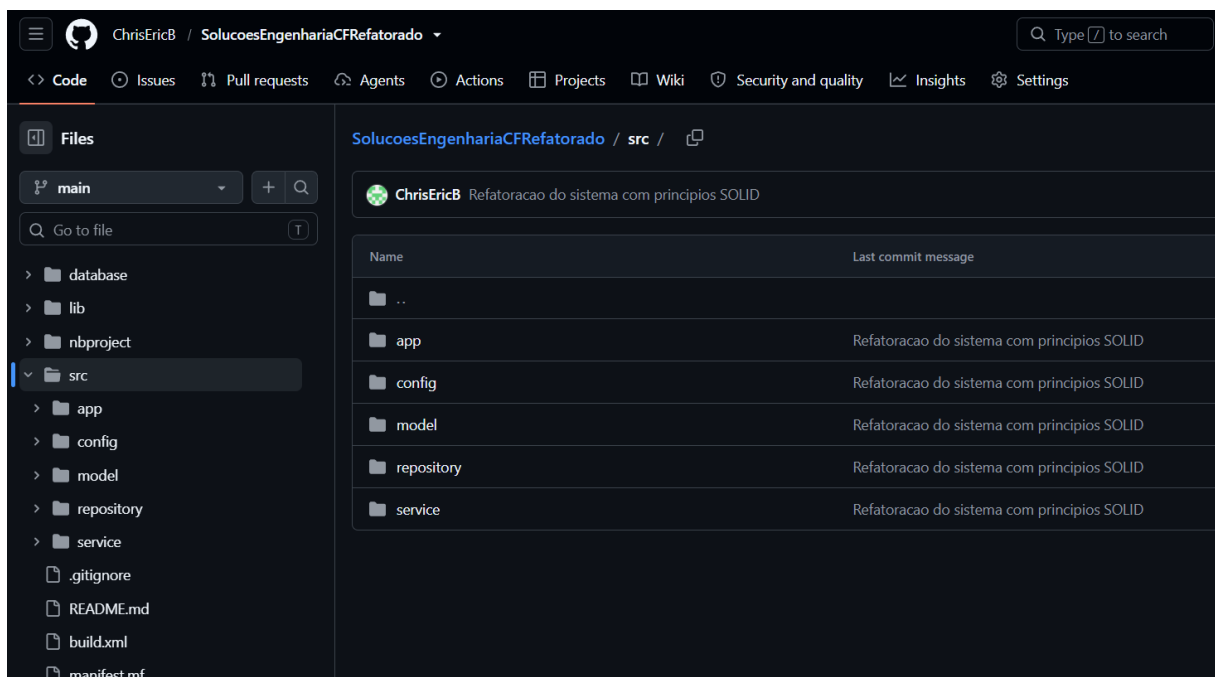


Figura 2 – Estrutura do diretório src no GitHub, com a separação dos pacotes.

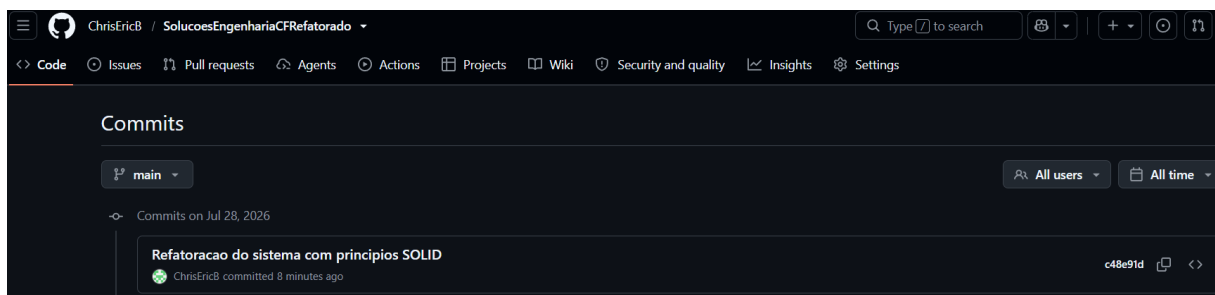


Figura 3 – Histórico do commit “Refatoracao do sistema com principios SOLID”.

9. Conclusão

A refatoração atingiu o objetivo de separar a interface desktop, as regras de negócio e o acesso aos dados. O novo projeto possui responsabilidades mais claras, menor acoplamento e melhor tratamento de erros. As interfaces de repositório e a injeção de dependência permitem que os serviços sejam reaproveitados na próxima fase web sem transportar componentes do Java Swing.

Os testes realizados confirmaram a conexão com o banco, a autenticação, as consultas das principais entidades e o funcionamento das validações. O código e as evidências foram registrados em um novo repositório GitHub, atendendo aos requisitos de versionamento e documentação da atividade.